Guía de resolución de todos los ejercicios de los PDF

Módulo: Desarrollo Web en Entorno Cliente (DAW2) **Documentos fuente:** `UD 2 - JavaScript Introduccion.pdf` y `UD 3 - JavaScript Objetos predefinidos.pdf` **Alcance:** todos los ejercicios de los enunciados, sus soluciones completas, explicación, errores típicos y variaciones posibles. **Restricción:** la guía usa solo conceptos, objetos y métodos que aparecen en la teoría de las UD2 y UD3. La UD1 no contiene ejercicios; solo teoría.

---

0. Cómo usar esta guía

1. Localiza el ejercicio en el índice de la sección 1.
2. Lee el enunciado resumido y el archivo que pide el profesor.
3. Copia la solución en el archivo indicado.
4. Prueba los casos de la sección 11.
5. Si el profesor cambia el enunciado, usa la sección 10: las variaciones se resuelven con los mismos objetos de la teoría.

Convenciones:

- `prompt()` devuelve `null` si el usuario cancela.
- `parseInt()` y `parseFloat()` devuelven `NaN` si el texto no es numérico. Se comprueba con `isNaN()`.
- `document.write()` solo se usa mientras la página se está cargando. Si se ejecuta después de cargar, borra toda la página.
- Para ejercicios con botón se usa `innerHTML` sobre un elemento, no `document.write`.
- Los nombres de archivo y de variables se mantienen como los pide el PDF, incluso si el PDF tiene erratas.
- Los eventos se escriben con atributos `onclick`, que sí aparecen en la teoría.

---

1. Inventario de ejercicios

ID	Documento	Página	Enunciado	Archivo pedido
2.12-1	UD2	11	Nombre, apellido, nombre completo, edad y año de nacimiento	`op1.js`
2.12-2	UD2	11	Exponencial de un número	`op1.js`
2.12-3	UD2	11	Par o impar con `%`	`parimpar.html`
2.12-4	UD2	11	Detectar múltiplo de otro número	`op1.js`
2.12-5	UD2	11	Array de 12 meses y bucle `for`	`op1.js`
2.12-6	UD2	11	Comparar dos enteros y comprobar positivos	`op1.js`
2.12-7	UD2	12	Mostrar los 30 primeros números	`op1.js`
2.12-8	UD2	12	Factorial con `for`	`factorial.html`
2.12-9	UD2	12	Ciudades con `switch`	`Elección multiple-respuesta.html`
2.13-1	UD2	13	Aviso con variable String `mensaje`	`op2.js`

ID	Documento	Página	Enunciado	Archivo pedido
2.13-2	UD2	13	División con desplazamiento a la derecha	`op2.js`
2.13-3	UD2	13	Multiplicación con desplazamiento a la izquierda	`op2.js`
2.13-4	UD2	13	Máximo, mínimo y valor infinito	`op2.js`
2.13-5	UD2	13	Comprobar mayúsculas, minúsculas o mixta	`op2.js`
2.13-A1	UD2	14	Media y calificación de la primera evaluación	aplicación Web
2.13-A2	UD2	14	Validar DNI con número y letra	`CalculoDNI` + `dni.js`
2.13-A3	UD2	15	Tablas de multiplicar, sumar, dividir y bits	`OperacionesBits` + `Operaciones`
3.12-1	UD3	19	Contador de clics con `localStorage`	página HTML
3.12-2	UD3	20	Contador de clics con `sessionStorage`	página HTML
3.13-1	UD3	21	Botón que vuelve a la página anterior	página HTML
3.13-2	UD3	21	Pop-up con una página web, centrado	página HTML
3.13-3	UD3	21	Del 1 al 500: múltiplos de 3 y no primos	página HTML
3.13-4	UD3	21	Menor y mayor de 10 números	página HTML
3.13-5	UD3	21	Talla europea a Grande/Mediana/Pequeña	página HTML
3.13-6	UD3	21	Longitud de la circunferencia	página HTML
3.13-7	UD3	22	Euromillones aleatorio	página HTML
3.13-8	UD3	22	Matriz de filas × columnas	página HTML
3.13-9	UD3	22	Calculadora completa	página HTML
3.13-10	UD3	23	Noticia con «Seguir leyendo»	página HTML
3.14-1	UD3	24	Tabla de 8 × 8 con patrón	`EjercicioAmpliacion1.7z`
3.14-2	UD3	24	Ejercicio de ampliación 2	`EjercicioAmpliacion2.7z`

2. Conceptos de la teoría que se usan

Esta sección es la lista de comprobación. Si una solución propuesta por el profesor usa algo que no está aquí, se puede resolver con un equivalente de esta tabla.

2.1. UD2: JavaScript básico

Tema	Elementos de la teoría
Variables	`var`, `let`, `const`; ámbito; hoisting
Tipos	`typeof`, `undefined`, `null`, String, Number, Boolean
Conversión	`Number()`, `parseInt()`, `parseFloat()`, `isNaN()`
Operadores	aritméticos, comparación, lógicos, `%`, `===`, `!==`, `==`, `!=`
Cadenas	comillas simples y dobles, concatenación con `+`, `length`, escapes `\n`, `\t`, `\"`, `\\`
Entrada y salida	`alert()`, `confirm()`, `prompt()`, `console.log()`
Control	`if`, `else`, `switch`, `case`, `default`, `break`

Tema	Elementos de la teoría
Bucles	`for`, `while`, `do...while`
Funciones	declaración `function nombre(parametros) { ... }`, `return`

2.2. UD3: objetos predefinidos

Objeto	Elementos de la teoría
`Date`	`new Date()`, `getDate()`, `getMonth()`, `getFullYear()`, `getDay()`
`Math`	`Math.exp()`, `Math.pow()`, `Math.random()`, `Math.floor()`, `Math.round()`, `Math.ceil()`, `Math.sqrt()`, `Math.PI`
`Number`	`Number.MAX_VALUE`, `Number.MIN_VALUE`, `Number.MAX_SAFE_INTEGER`, `Number.MIN_SAFE_INTEGER`, `Number.NaN`, `Number.POSITIVE_INFINITY`, `Number.NEGATIVE_INFINITY`, `toPrecision()`
`String`	`length`, `toString()`, `concat()`, `toUpperCase()`, `toLowerCase()`, `indexOf()`, `lastIndexOf()`, `substring()`, `split()`, `replace()`, `charAt()`, `charCodeAt()`
`Array`	`new Array()`, literal `[]`, `length`, acceso por índice, `push()`, `pop()`
`Boolean`	constructor, `toString()`, `valueOf()`
`Object`	literal `{}`, `Object.keys()`, `Object.values()`, `Object.entries()`, `Object.assign()`, `Object.freeze()`
`JSON`	`JSON.stringify()`, `JSON.parse()`
`Promise`	`then()`, `catch()`, `finally()`, `Promise.all()`, `Promise.race()`
`Map` y `Set`	`new Map()`, `set()`, `get()`; `new Set([...])`
Control	`if/else`, `switch`, `for`, `while`, `do...while`, `for...in`, `for...of`, `forEach`, `break`, `continue`, `return`
`Navigator`	`appCodeName`, `appName`, `appVersion`, `cookieEnabled`, `platform`, `userAgent`, `plugins`
`Screen`	`availHeight`, `availWidth`, `colorDepth`, `height`, `pixelDepth`, `width`
`History`	`current`, `length`, `next`, `previous`, `back()`, `forward()`, `go(posicion)`
`Location`	`hash`, `hostname`, `pathname`, `protocol`, `host`, `href`, `port`, `search`, `reload()`, `replace()`, `assign()`
`Window`	`document`, `location`, `navigator`, `screen`, `history`, `innerWidth`, `innerHeight`, `localStorage`, `sessionStorage`, `alert`, `confirm`, `prompt`, `open`, `close`, `setTimeout`, `clearTimeout`, `setInterval`, `clearInterval`, `scrollTo`
`Document`	`getElementById()`, `getElementsByName()`, `getElementsByTagName()`, `write()`, `writeln()`, `open()`, `close()`, `forms[]`, `images[]`, `links[]`, `title`, `URL`
Almacenamiento	`localStorage`, `sessionStorage`; acceso por propiedad, `setItem()`, `getItem()`, `removeItem()`, `clear()`, `typeof(Storage)`

2.3. Equivalencias cuando el enunciado sugiere algo que no está en la teoría

Si el enunciado o un ejemplo usa...	En esta guía se usa...
Escuchadores de eventos modernos	`onclick="f()"` en el HTML
Métodos modernos para escribir texto	`innerHTML`
Métodos modernos para crear nodos o filas	`document.write()` o una cadena con `innerHTML`
Funciones modernas de limpieza de texto	Comprobación de cadena vacía con `length`; si hace falta, un bucle con `charAt()`
Formato decimal moderno	`Math.round(valor * 100) / 100`
Búsqueda moderna dentro de un array	Un bucle `for` que compare con `===`, o `indexOf() !== -1`

Si el enunciado o un ejemplo usa...	En esta guía se usa...
Ordenación moderna de arrays	Un bucle manual de ordenación, o no ordenar
Funciones modernas de mínimo, máximo o valor absoluto	Comparaciones con `if`; para el valor absoluto, `if (n < 0) n = -n;`
Comprobaciones modernas de número	`isNaN()`, `Math.floor(n) === n`, comparaciones de rango
Expresiones regulares	`length`, `charAt()`, `charCodeAt()`, `substring()`
Plantillas de cadena y funciones flecha	Concatenación con `+` y funciones `function`

---

3. Plantillas base

Para los ejercicios que piden un archivo `.js` separado:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio</title>
</head>
<body>
  <div id="resultado"></div>
  <script src="op1.js"></script>
</body>
</html>
```

Para los ejercicios con botón:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejercicio</title>
</head>
<body>
  <button onclick="ejecutar()">Ejecutar</button>
  <div id="resultado"></div>

  <script>
    function ejecutar() {
      document.getElementById("resultado").innerHTML = "Resultado";
    }
  </script>
</body>
</html>
```

Regla práctica:

- `prompt` + `alert`: ejercicios de consola o avisos.
- `prompt` + `innerHTML`: ejercicios de aplicación Web.
- `document.write`: solo si el enunciado pide «mostrar en el documento HTML» y el script se ejecuta durante la carga.
- El script se coloca al final del `body` para que los elementos existan cuando se usa `getElementById`.

---

4. UD2 — Fichero `op1.js` (ejercicios 2.12)

4.1. Ejercicio 1: nombre, apellido, edad y año de nacimiento

Enunciado. Pedir nombre y primer apellido. Guardarlos en `nombre` y `apellido`. Crear `fullName` con ambos separados por un espacio. Pedir la edad en `edad`. Calcular el año de nacimiento en `year` (sin mes). Mostrar «Nombre completo» y «Año de nacimiento», uno por línea.

Solución:

```
let nombre = prompt("Introduce tu nombre:");
let apellido = prompt("Introduce tu primer apellido:");
let fullName = nombre + " " + apellido;

let edad = parseInt(prompt("Introduce tu edad:"), 10);
let year = new Date().getFullYear() - edad;

document.write("Nombre completo: " + fullName + "<br>");
document.write("Año de nacimiento: " + year + "<br>");
```

Explicación:

- `prompt` devuelve una cadena; para la edad se convierte con `parseInt(..., 10)`.
- `new Date().getFullYear()` devuelve el año actual. El cálculo no tiene en cuenta el mes, tal como pide el enunciado.
- El enunciado menciona una variable `Nacimiento` y luego muestra `year`. Se puede usar `let Nacimiento = year;` para cumplir literalmente las dos.

Variante con validación:

```
let nombre = prompt("Nombre:");
let apellido = prompt("Primer apellido:");
let edad = parseInt(prompt("Edad:"), 10);

if (nombre === null || apellido === null || nombre.length === 0 ||
    apellido.length === 0 || isNaN(edad) || edad < 0) {
  document.getElementById("resultado").innerHTML =
    "Datos no válidos. Introduce nombre, apellido y una edad entera positiva.";
} else {
  let fullName = nombre + " " + apellido;
  let year = new Date().getFullYear() - edad;
  document.getElementById("resultado").innerHTML =
    "Nombre completo: " + fullName + "<br>Año de nacimiento: " + year;
}
```

Otras variantes: calcular el año exacto con mes y día; usar `const ANIO_ACTUAL`; mostrar también la edad; guardar el resultado en `localStorage`.

4.2. Ejercicio 2: exponencial de cualquier número

Enunciado. Calcular el exponencial de cualquier número.

Solución:

```
let numero = parseFloat(prompt("Introduce un número:"));

if (isNaN(numero)) {
  document.write("El valor introducido no es un número.");
} else {
  document.write("e^" + numero + " = " + Math.exp(numero));
}
```

Explicación: `Math.exp(x)` calcula \(e^x\). No es lo mismo que `Math.pow(base, exponente)`, que calcula una potencia con base elegida.

Variantes:

```
// Potencia con base y exponente introducidos por teclado
let base = parseFloat(prompt("Base:"));
let exponente = parseFloat(prompt("Exponente:"));
document.write(base + "^" + exponente + " = " + Math.pow(base, exponente));
```

Otras variantes: mostrar 2 decimales con `Math.round(resultado * 100) / 100`; comprobar que el resultado no es `Infinity` con `resultado === Infinity`; usar `Math.exp(numero)` con números negativos.

4.3. Ejercicio 3: par o impar (`parimpar.html`)

Enunciado. Calcular si un número introducido por teclado es par o impar usando el módulo `%`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Par o impar</title>
</head>
<body>
  <script>
    let numero = parseInt(prompt("Introduce un número entero:"), 10);

    if (isNaN(numero)) {
      document.write("No has introducido un número entero.");
    } else if (numero % 2 === 0) {
      document.write("El número " + numero + " es par.");
    } else {
      document.write("El número " + numero + " es impar.");
    }
  </script>
</body>
</html>
```

Explicación: `numero % 2` devuelve `0` si es divisible por 2. Con números negativos, JavaScript devuelve `-1` para `-3 % 2`, por eso se compara con `0`, no con `1`.

Variantes:

```
// Operador ternario
let paridad = numero % 2 === 0 ? "par" : "impar";

// Negativos: convertir a positivo con un if
let n = numero;
if (n < 0) { n = -n; }
let paridad2 = n % 2 === 0 ? "par" : "impar";
```

Otras variantes: pedir varios números en bucle; contar pares e impares; usar `input` en lugar de `prompt`; validar que sea entero con `Math.floor(numero) === numero`.

4.4. Ejercicio 4: detectar si un número es múltiplo de otro

Enunciado. Detectar si un número es múltiplo de otro.

Solución:

```
let numero = parseInt(prompt("Introduce el número:"), 10);
let divisor = parseInt(prompt("Introduce el posible divisor:"), 10);

if (isNaN(numero) || isNaN(divisor)) {
  document.write("Introduce dos números enteros.");
} else if (divisor === 0) {
  document.write("No se puede dividir entre cero.");
} else if (numero % divisor === 0) {
  document.write(numero + " es múltiplo de " + divisor + ".");
} else {
  document.write(numero + " no es múltiplo de " + divisor + ".");
}
```

Explicación: `a` es múltiplo de `b` si `a % b === 0`. Hay que comprobar `b !== 0` antes de usar el módulo.

Variantes:

```
// Comprobar ambas direcciones
let aEsMultiploDeB = numero % divisor === 0;
let bEsMultiploDeA = divisor % numero === 0;
```

Otras variantes: mostrar el cociente exacto; comprobar si es divisible por 3 y 5 a la vez; listar los múltiplos de un número hasta un límite.

4.5. Ejercicio 5: array de los 12 meses con `for`

Enunciado. Crear un array con los 12 meses y mostrar cada mes con un `for`.

Solución:

```
let meses = [
  "Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
  "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"
];

for (let i = 0; i < meses.length; i++) {
  document.write((i + 1) + ". " + meses[i] + "<br>");
}
```

Explicación: El array empieza en el índice `0`. El número de mes se muestra con `i + 1`.

Variantes:

```
// for...of
for (let mes of meses) {
  document.write(mes + "<br>");
}

// Recorrido inverso
for (let i = meses.length - 1; i >= 0; i--) {
  document.write(meses[i] + "<br>");
}
```

Otras variantes: mostrar solo los meses con 31 días; crear el array con `new Array(...)`; pedir un número de mes y devolver su nombre; añadir numeración con un contador.

4.6. Ejercicio 6: comparar dos enteros y comprobar positivos

Enunciado. Comparar dos números enteros para saber cuál es mayor. Además, comprobar si son positivos.

Solución:

```
let a = parseInt(prompt("Primer número entero:"), 10);
let b = parseInt(prompt("Segundo número entero:"), 10);

if (isNaN(a) || isNaN(b)) {
  document.write("Introduce dos números enteros.");
} else {
  if (a > b) {
    document.write(a + " es mayor que " + b + ".<br>");
  } else if (b > a) {
    document.write(b + " es mayor que " + a + ".<br>");
  } else {
    document.write("Los dos números son iguales.<br>");
  }

  if (a > 0 && b > 0) {
    document.write("Los dos son positivos.");
  } else if (a > 0 || b > 0) {
    document.write("Solo uno es positivo.");
  } else {
    document.write("Ninguno es positivo.");
  }
}
```

Explicación: `>` compara valores numéricos. `0` no es positivo; para incluirlo habría que usar `>= 0`. Se distingue «los dos», «solo uno» y «ninguno».

Variantes:

```
// Indicar cuál es positivo y cuál no
let estadoA = a > 0 ? "positivo" : "no positivo";
let estadoB = b > 0 ? "positivo" : "no positivo";
document.write("a es " + estadoA + " y b es " + estadoB + ".");
```

Otras variantes: comparar tres números; ordenar de mayor a menor con comparaciones; comprobar paridad además de positividad; mostrar el valor absoluto con `if`.

4.7. Ejercicio 7: mostrar los 30 primeros números

Enunciado. Mostrar en el documento HTML los 30 primeros números.

Solución:

```
for (let i = 1; i <= 30; i++) {
  document.write(i + (i < 30 ? ", " : "."));
}
```

Explicación: El bucle va de 1 a 30, ambos incluidos. La condición `i < 30` evita la coma final.

Variantes:

```
// Del 0 al 29
for (let i = 0; i < 30; i++) {
  document.write(i + "<br>");
}

// Solo pares
for (let i = 2; i <= 60; i += 2) {
  document.write(i + "<br>");
}

// En una tabla
document.write("<table border='1'>");
for (let i = 1; i <= 30; i++) {
  document.write("<tr><td>" + i + "</td><td>" + (i * i) + "</td></tr>");
}
document.write("</table>");
```

Otras variantes: con `while`; con `do...while`; sumar los 30 primeros números; mostrar sus cuadrados; mostrar en orden inverso.

4.8. Ejercicio 8: factorial con `for` (`factorial.html`)

Enunciado. Mostrar el factorial de un número introducido por teclado. Declarar dos variables: una para el número y otra para el resultado. Emplear un `for`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Factorial</title>
</head>
<body>
  <script>
    let numero = parseInt(prompt("Introduce un número entero no negativo:"), 10);
    let resultado = 1;

    if (isNaN(numero) || numero < 0) {
      document.write("El número debe ser un entero no negativo.");
    } else {
      for (let i = 2; i <= numero; i++) {
        resultado = resultado * i;
      }
      document.write(numero + "! = " + resultado);
    }
  </script>
</body>
</html>
```

Explicación: El factorial de 0 es 1. El bucle empieza en 2 porque multiplicar por 1 no cambia el resultado. Para números grandes se pierde precisión; `BigInt` aparece en la teoría y se puede usar si el profesor lo pide.

Variantes:

```
// Con BigInt para números grandes
let resultado = 1n;
for (let i = 2n; i <= BigInt(numero); i++) {
  resultado = resultado * i;
}
document.write(numero + "! = " + resultado);
```

Otras variantes: factorial con `while`; factorial recursivo; comprobar si el número es demasiado grande; mostrar la operación completa (`5! = 5 × 4 × 3 × 2 × 1 = 120`).

4.9. Ejercicio 9: ciudades con `switch` (`Elección multiple-respuesta.html`)

Enunciado. Al pulsar un botón, pedir una ciudad. Mostrar una elección múltiple de 3 ciudades con `switch`: Zaragoza, Barcelona y Madrid, cada una con su mensaje. Fuera de las opciones, mostrar «No conozco esa ciudad».

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Elección múltiple</title>
</head>
<body>
  <button onclick="elegirCiudad()">Elegir ciudad</button>
  <div id="resultado"></div>

  <script>
    function elegirCiudad() {
      let entrada = prompt("¿Cuál es tu ciudad favorita? (Zaragoza, Barcelona, Madrid)");
      let resultado = document.getElementById("resultado");

      if (entrada === null) {
        resultado.innerHTML = "Operación cancelada.";
        return;
      }

      let ciudad = entrada.toLowerCase();

      switch (ciudad) {
        case "zaragoza":
          resultado.innerHTML = "Maravillosa gente. Capital del Ebro.";
          break;
        case "barcelona":
          resultado.innerHTML = "Ciudad cosmopolita, modernista y mediterránea.";
          break;
        case "madrid":
          resultado.innerHTML = "Capital de España, con mucha vida cultural.";
          break;
        default:
          resultado.innerHTML = "No conozco esa ciudad.";
      }
    }
  </script>
</body>
</html>
```

Explicación: Se usa `toLowerCase()` para que `Zaragoza` y `zaragoza` funcionen igual. El `default` cubre cualquier otra ciudad y el caso de cadena vacía. El mensaje de Zaragoza es el que aparece en el PDF; los de Barcelona y Madrid son ejemplos adaptables.

Variante que quita espacios con la teoría:

```
function quitarEspacios(texto) {
  let salida = "";
  for (let i = 0; i < texto.length; i++) {
    let c = texto.charAt(i);
    if (c !== " ") {
      salida = salida + c;
    }
  }
  return salida;
}

let ciudad = quitarEspacios(entrada).toLowerCase();
```

Otras variantes: con `if/else`; con un objeto de mensajes; usar `select` con tres opciones; mostrar la elección en un `alert`; pedir la ciudad hasta que sea válida; añadir más ciudades.

---

5. UD2 — Fichero `op2.js` (ejercicios 2.13)

5.1. Ejercicio 1: aviso con variable String `mensaje`

Enunciado. Mostrar un aviso mediante JavaScript. Crear una variable de tipo String llamada `mensaje`. El aviso del PDF contiene:

```
Esta página dice
Hola a todo el Mundo!
Qué facil es incluir 'comillas simples'
y "comillas dobles"
```

Solución:

```
let mensaje = "Esta página dice\n" +
              "Hola a todo el Mundo!\n" +
              "Qué facil es incluir 'comillas simples'\n" +
              'y "comillas dobles"';

alert(mensaje);
```

Explicación: `\n` introduce un salto de línea. Las comillas simples y dobles se pueden incluir si se elige el delimitador adecuado o se escapan con `\`.

Variantes:

```
// Con comillas dobles escapadas
let mensaje2 = "Esta página dice\nHola a todo el Mundo!\n" +
               "Qué facil es incluir 'comillas simples'\n" +
               "y \"comillas dobles\"";
alert(mensaje2);

// En un div, con <br> en lugar de \n
document.getElementById("resultado").innerHTML =
  "Esta página dice<br>Hola a todo el Mundo!<br>" +
  "Qué facil es incluir 'comillas simples'<br>y \"comillas dobles\"";
```

Otras variantes: usar `console.log`; usar `confirm` para pedir aceptar/cancelar; concatenar varias variables.

5.2. Ejercicio 2: división con desplazamiento a la derecha

Enunciado. Hacer divisiones en binario con desplazamiento a la derecha. Ejemplo: decimal `40 / 16` frente a binario `40 >> 4`.

Solución:

```
let numero = parseInt(prompt("Introduce el dividendo:"), 10);
let desplazamiento = parseInt(prompt("Introduce el desplazamiento:"), 10);

if (isNaN(numero) || isNaN(desplazamiento) ||
    desplazamiento < 0 || desplazamiento > 31) {
  document.write("Introduce un entero y un desplazamiento entre 0 y 31.");
} else {
  let divisor = Math.pow(2, desplazamiento);
  document.write("Decimal: " + numero + " / " + divisor + " = " +
                 (numero / divisor) + "<br>");
  document.write("Binario: " + numero + " >> " + desplazamiento + " = " +
                 (numero >> desplazamiento));
}
```

Explicación: `x >> n` equivale a dividir entre \(2^n\) y quedarse con la parte entera. Por eso `40 >> 4` da `2`, mientras que `40 / 16` da `2.5`. No es una división decimal exacta: trunca.

Ejemplos que deben coincidir:

División decimal	Desplazamiento	Resultado decimal	Resultado binario
`40 / 16`	`40 >> 4`	`2,5`	`2`
`125 / 8`	`125 >> 3`	`15,625`	`15`
`25 / 2`	`25 >> 1`	`12,5`	`12`

Variantes: comprobar el resultado con `Math.floor(numero / Math.pow(2, desplazamiento))`; mostrar el número en binario con una función que divida entre 2; usar `BigInt` con `>>` para números grandes.

5.3. Ejercicio 3: multiplicación con desplazamiento a la izquierda

Enunciado. Hacer multiplicaciones en binario con desplazamiento a la izquierda. Ejemplo: decimal `26 × 4` frente a binario `26 << 2`.

Solución:

```
let numero = parseInt(prompt("Introduce el número:"), 10);
let desplazamiento = parseInt(prompt("Introduce el desplazamiento:"), 10);

if (isNaN(numero) || isNaN(desplazamiento) ||
    desplazamiento < 0 || desplazamiento > 31) {
  document.write("Introduce un entero y un desplazamiento entre 0 y 31.");
} else {
  let multiplicador = Math.pow(2, desplazamiento);
  document.write("Decimal: " + numero + " x " + multiplicador + " = " +
                 (numero * multiplicador) + "<br>");
  document.write("Binario: " + numero + " << " + desplazamiento + " = " +
                 (numero << desplazamiento));
}
```

Explicación: `x << n` equivale a multiplicar por \(2^n\), siempre que no se desborde el entero de 32 bits. `26 << 2` es `104`, igual que `26 × 4`.

Ejemplos que deben coincidir:

Multiplicación decimal	Desplazamiento	Resultado
`26 × 4`	`26 << 2`	`104`
`40 × 4`	`40 << 2`	`160`
`10 × 16`	`10 << 4`	`160`

Variantes: multiplicar por 8 con `<< 3`; dividir por 4 con `>> 2`; comprobar el desbordamiento comparando con `2147483647`; usar `BigInt` para evitar el límite de 32 bits.

5.4. Ejercicio 4: máximo, mínimo y valor infinito

Enunciado. Mostrar por consola el máximo y el valor más cercano a cero posible en JavaScript. Ayuda: `Number.MAX_VALUE` y `Number.MIN_VALUE`. Después, mostrar un valor infinito.

Solución:

```
console.log("Máximo valor representable:", Number.MAX_VALUE);
console.log("Mínimo valor positivo representable:", Number.MIN_VALUE);
console.log("Valor más negativo:", -Number.MAX_VALUE);
console.log("Infinito positivo:", Number.POSITIVE_INFINITY);
console.log("Infinito negativo:", Number.NEGATIVE_INFINITY);
console.log("Infinito generado con 1 / 0:", 1 / 0);
console.log("Infinito por desbordamiento:", Number.MAX_VALUE * 2);
```

Explicación importante: `Number.MIN_VALUE` **no** es el número más negativo; es el número positivo más pequeño distinto de cero (aproximadamente \(5 \times 10^{-324}\)). El más negativo es `-Number.MAX_VALUE`. `Number.MAX_SAFE_INTEGER` y `Number.MIN_SAFE_INTEGER` son otros límites distintos.

Variantes:

```
// Máximo entero seguro
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
console.log(Number.MIN_SAFE_INTEGER); // -9007199254740991

// Comparar con Infinity
let infinito = Number.POSITIVE_INFINITY;
console.log(infinito === Infinity); // true
console.log(isNaN(infinito));       // false
```

Otras variantes: sumar 1 a `Number.MAX_VALUE`; comparar `Infinity` con `Number.POSITIVE_INFINITY`; mostrar `-Infinity`; comprobar `Number.NaN` con `isNaN()`.

5.5. Ejercicio 5: mayúsculas, minúsculas o ambas

Enunciado. Dadas `OVNI = "OBJETO VOLADOR NO IDENTIFICADO"` e `Info = "En un lugar de la mancha"`, comprobar si están en minúsculas, mayúsculas o ambas. Emplear `toUpperCase()` y `toLowerCase()`. Ampliar para evaluar cualquier cadena introducida por el usuario.

Solución:

```
function tipoDeCadena(texto) {
  let enMayusculas = texto === texto.toUpperCase();
  let enMinusculas = texto === texto.toLowerCase();

  if (enMayusculas && enMinusculas) {
    return "No contiene letras o no se puede determinar.";
  }
  if (enMayusculas) {
    return "Está en mayúsculas.";
  }
  if (enMinusculas) {
    return "Está en minúsculas.";
  }
  return "Es mixta (mayúsculas y minúsculas).";
}

let OVNI = "OBJETO VOLADOR NO IDENTIFICADO";
let Info = "En un lugar de la mancha";

console.log("OVNI:", tipoDeCadena(OVNI));
console.log("Info:", tipoDeCadena(Info));

let usuario = prompt("Introduce una cadena:");
console.log("Usuario:", tipoDeCadena(usuario));
```

Explicación: Una cadena está en mayúsculas si es igual a su versión en mayúsculas. Está en minúsculas si es igual a su versión en minúsculas. Si no cambia con ninguna de las dos, es que no tiene letras (por ejemplo, `"123"`). Si cambia con las dos, es mixta.

Variante para contar mayúsculas y minúsculas:

```
function contarLetras(texto) {
  let mayusculas = 0;
  let minusculas = 0;

  for (let i = 0; i < texto.length; i++) {
    let c = texto.charAt(i);
    if (c >= "A" && c <= "Z") {
      mayusculas++;
    } else if (c >= "a" && c <= "z") {
      minusculas++;
    }
  }

  return "Mayúsculas: " + mayusculas + ". Minúsculas: " + minusculas + ".";
}
```

Otras variantes: ignorar espacios y signos; comprobar si empieza por mayúscula; contar palabras con `split(" ")`; comparar la cadena con su versión invertida.

---

6. UD2 — Ejercicios de ampliación (2.13)

6.1. Ampliación 1: media y calificación

Enunciado. Pedir `NotaProyecto` y `NotaExamen` (0 a 10). Calcular la media y mostrar la calificación:

- Si alguna nota es menor de 4,5: Suspenso.
- Si ambas son mayores o iguales que 4,5 pero la media es menor que 5: Suspenso.
- Media entre 5 y menor que 7: Aprobado.
- Media entre 7 y menor que 9: Notable.
- Media mayor o igual que 9: Sobresaliente.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Nota de evaluación</title>
</head>
<body>
  <div id="resultado"></div>
  <script>
    let NotaProyecto = parseFloat(prompt("Nota del proyecto (0-10):"));
    let NotaExamen = parseFloat(prompt("Nota del examen (0-10):"));

    function notaValida(n) {
      return !isNaN(n) && n >= 0 && n <= 10;
    }

    if (!notaValida(NotaProyecto) || !notaValida(NotaExamen)) {
      document.getElementById("resultado").innerHTML =
        "Las notas deben ser números entre 0 y 10.";
    } else {
      let media = (NotaProyecto + NotaExamen) / 2;
      let calificacion;

      if (NotaProyecto < 4.5 || NotaExamen < 4.5) {
        calificacion = "Suspenso";
      } else if (media < 5) {
        calificacion = "Suspenso";
      } else if (media < 7) {
        calificacion = "Aprobado";
      } else if (media < 9) {
        calificacion = "Notable";
      } else {
        calificacion = "Sobresaliente";
      }

      document.getElementById("resultado").innerHTML =
        "Media: " + (Math.round(media * 100) / 100) +
        "<br>Calificación: " + calificacion;
    }
  </script>
</body>
</html>
```

Explicación: Las dos primeras ramas pueden unirse, pero se mantienen separadas para reflejar literalmente los casos del enunciado. Los límites son: 5 inclusive para Aprobado, 7 inclusive para Notable y 9 inclusive para Sobresaliente. La media se calcula después de validar.

Casos de prueba:

Proyecto	Examen	Media	Resultado
4,4	10	7,2	Suspenso (nota < 4,5)
4,5	4,5	4,5	Suspenso (media < 5)
4,5	5,5	5,0	Aprobado
7,0	7,0	7,0	Notable
8,9	9,1	9,0	Sobresaliente
10	10	10	Sobresaliente
-1	5	—	Error de validación

Variantes: media ponderada (proyecto 40 %, examen 60 %); redondear la media antes de clasificar; usar `switch (true)`; pedir las notas con `input`; mostrar la nota con un decimal; añadir «Matrícula de honor» a partir de 9,5.

6.2. Ampliación 2: validar DNI (`CalculoDNI` + `dni.js`)

Enunciado. Pedir número y letra del DNI. Comprobar si es correcto. Excepciones: longitud del número incorrecta, letra no introducida, carácter de la letra incorrecto. El archivo JS se llama `dni.js`; la aplicación, `CalculoDNI`. Crear un array llamado `abcedario` con:

```
['T','R','W','A','G','M','Y','F','P','D','X','B','N','J','Z','S','Q','V','H','L','C','K','E']
```

El número no puede ser negativo ni mayor que 99.999.999. La letra se calcula con el módulo del número entre las posiciones del array.

Solución `dni.js`:

```
let abcedario = [
  'T','R','W','A','G','M','Y','F','P','D','X','B','N','J','Z',
  'S','Q','V','H','L','C','K','E'
];

function soloDigitos(texto) {
  if (texto.length === 0) {
    return false;
  }
  for (let i = 0; i < texto.length; i++) {
    let codigo = texto.charCodeAt(i);
    if (codigo < 48 || codigo > 57) {
      return false;
    }
  }
  return true;
}

function validarDNI(numeroTexto, letraTexto) {
  if (numeroTexto.length === 0) {
    return "El número de DNI no puede estar vacío.";
  }
  if (letraTexto.length === 0) {
```

```
    return "La letra del DNI no puede estar vacía.";
  }
  if (numeroTexto.length > 8) {
    return "La longitud del número es incorrecta: máximo 8 dígitos.";
  }
  if (!soloDigitos(numeroTexto)) {
    return "El número debe contener solo dígitos.";
  }
  if (letraTexto.length !== 1) {
    return "La letra debe ser un único carácter.";
  }

  let codigoLetra = letraTexto.toUpperCase().charCodeAt(0);
  if (codigoLetra < 65 || codigoLetra > 90) {
    return "La letra debe ser un carácter alfabético.";
  }

  let valor = parseInt(numeroTexto, 10);
  if (valor < 0 || valor > 99999999) {
    return "El número de DNI no existe: debe estar entre 0 y 99.999.999.";
  }

  let letraEsperada = abcedario[valor % abcedario.length];
  if (letraTexto.toUpperCase() !== letraEsperada) {
    return "DNI incorrecto. La letra correcta para " + valor +
           " es " + letraEsperada + ".";
  }
  return "DNI correcto: " + numeroTexto + letraEsperada + ".";
}
```

Solución `CalculoDNI.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>CalculoDNI</title>
</head>
<body>
  <label>Número: <input id="numero" type="text"></label><br>
  <label>Letra: <input id="letra" type="text" maxlength="1"></label><br>
  <button onclick="comprobar()">Comprobar DNI</button>
  <p id="resultado"></p>

  <script src="dni.js"></script>
  <script>
    function comprobar() {
      let numero = document.getElementById("numero").value;
      let letra = document.getElementById("letra").value;
      document.getElementById("resultado").innerHTML = validarDNI(numero, letra);
    }
  </script>
</body>
</html>
```

Explicación: El array tiene 23 letras. `valor % 23` devuelve una posición entre 0 y 22. Se usa `% abcedario.length` para no depender del número literal. La letra se compara en mayúsculas para aceptar minúsculas. `soloDigitos` recorre la cadena con `charCodeAt` y comprueba que cada código esté entre 48 (`0`) y 57 (`9`). No se usan expresiones regulares porque no aparecen en la teoría.

Ejemplos válidos: `12345678Z`, `87654321X`, `00000000T`.

Casos de prueba:

Número	Letra	Resultado
`12345678`	`Z`	Correcto
`12345678`	`z`	Correcto
`12345678`	`A`	Incorrecto; letra correcta Z
`1234567`	`L`	Correcto si se aceptan 1-8 dígitos
`123456789`	`Z`	Longitud incorrecta
`-1234567`	`L`	Carácter incorrecto en el número
`12345678`	(vacío)	Letra no introducida
`12345678`	`1`	Carácter de letra incorrecto
`99999999`	`R`	Correcto
`100000000`	`R`	Número inexistente

Variantes: aceptar exactamente 8 dígitos cambiando la comprobación por `numeroTexto.length !== 8`; calcular la letra automáticamente y compararla; mostrar la letra esperada siempre; separar la lógica en funciones `validarNumero`, `validarLetra`, `calcularLetra`.

6.3. Ampliación 3: `OperacionesBits` con `Operaciones.js`

Enunciado. Crear una aplicación Web que muestre:

- Tabla de multiplicar del 7.
- Tabla de sumar del 8.
- Tabla de dividir del 9.
- Ejemplos de desplazamiento de bits: `125 / 8` → `125 >> 3 = 15`; `25 / 2` → `25 >> 1 = 12`; `40 × 4` → `40 << 2 = 160`; `10 × 16` → `10 << 4 = 160`.

Condiciones:

- La aplicación se llama `OperacionesBits`.
- El archivo JavaScript se llama `Operaciones`.
- Usar los tres tipos de bucles de JavaScript, un bucle distinto para cada número.
- No borrar ninguna línea del código original.
- No cambiar los nombres de los archivos.
- Escribir el código debajo de cada comentario.

Solución `OperacionesBits.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>OperacionesBits</title>
</head>
<body>
  <h1>OperacionesBits</h1>
  <div id="resultado"></div>
  <script src="Operaciones.js"></script>
</body>
</html>
```

Solución `Operaciones.js`:

```
let tabla7 = [];
for (let i = 1; i <= 10; i++) {
  tabla7.push("7 x " + i + " = " + (7 * i));
}

let tabla8 = [];
let i = 1;
while (i <= 10) {
  tabla8.push("8 + " + i + " = " + (8 + i));
  i++;
}

let tabla9 = [];
i = 1;
do {
  tabla9.push("9 / " + i + " = " + (9 / i));
  i++;
} while (i <= 10);

let html = "<table border='1' cellpadding='4'>";
html += "<tr><th>Multiplicar por 7</th><th>Sumar 8</th><th>Dividir entre 9</th></tr>";

for (let fila = 0; fila < 10; fila++) {
  html += "<tr>";
  html += "<td>" + tabla7[fila] + "</td>";
  html += "<td>" + tabla8[fila] + "</td>";
  html += "<td>" + tabla9[fila] + "</td>";
  html += "</tr>";
}

html += "</table>";
html += "<h2>Desplazamiento de bits</h2>";
html += "<p>125 / 8 = " + (125 / 8) + "; 125 >> 3 = " + (125 >> 3) + "</p>";
html += "<p>25 / 2 = " + (25 / 2) + "; 25 >> 1 = " + (25 >> 1) + "</p>";
html += "<p>40 x 4 = " + (40 * 4) + "; 40 << 2 = " + (40 << 2) + "</p>";
html += "<p>10 x 16 = " + (10 * 16) + "; 10 << 4 = " + (10 << 4) + "</p>";

document.getElementById("resultado").innerHTML = html;
```

Explicación: Cada tabla se genera con un tipo de bucle distinto: `for` para el 7, `while` para el 8 y `do...while` para el 9. Después se recorren los tres arrays con un `for` para construir las filas. Los desplazamientos son operaciones enteras: `125 >> 3` da `15` porque descarta la parte decimal; `25 >> 1` da `12`; `40 << 2` da `160`; `10 << 4` da `160`.

Si el archivo original ya tiene líneas y comentarios: no las borres. Pega el bucle correspondiente debajo de cada comentario y conserva el resto. Si el archivo original declara arrays o variables con otros nombres, adapta las referencias sin renombrar el archivo.

Variantes: usar `document.write` en lugar de `innerHTML`; usar `for...of` para recorrer los arrays; añadir la tabla del 6 con `for` anidado; mostrar los decimales con `Math.round(valor * 100) / 100`; comprobar los

desplazamientos con `Math.floor` y `Math.pow`.

---

7. UD3 — Almacenamiento Web (3.12)

7.1. Ejercicio 1: contador con `localStorage`

Enunciado. Crear una función para contar las veces que se hace clic a un botón. El número debe quedar almacenado y conservarse al cerrar el navegador.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Contador localStorage</title>
</head>
<body>
  <button onclick="clickCounter()">Haz clic</button>
  <p id="resultado"></p>
  <button onclick="reiniciar()">Reiniciar contador</button>

  <script>
    function clickCounter() {
      if (typeof(Storage) !== "undefined") {
        if (localStorage.clickcount) {
          localStorage.clickcount = Number(localStorage.clickcount) + 1;
        } else {
          localStorage.clickcount = 1;
        }
        document.getElementById("resultado").innerHTML =
          "Has hecho clic " + localStorage.clickcount +
          " veces desde la primera vez.";
      } else {
        document.getElementById("resultado").innerHTML =
          "Tu navegador no soporta localStorage.";
      }
    }

    function reiniciar() {
      localStorage.removeItem("clickcount");
      document.getElementById("resultado").innerHTML = "Contador reiniciado.";
    }
  </script>
</body>
</html>
```

Explicación: `localStorage` no expira al cerrar el navegador. El PDF usa el acceso por propiedad (`localStorage.clickcount`). `removeItem("clickcount")` borra esa misma clave. `Number()` convierte el

valor guardado, que siempre es una cadena, a número.

Variantes: usar `setItem("clickcount", valor)` y `getItem("clickcount")`; guardar varios contadores con claves distintas; guardar un objeto con `JSON.stringify` / `JSON.parse`; mostrar la fecha del último clic con `new Date()`; usar `localStorage.clear()` con confirmación.

7.2. Ejercicio 2: contador con `sessionStorage`

Enunciado. Crear la misma función, pero que el contador se inicie a 0 cuando se cierre la sesión del navegador.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Contador sessionStorage</title>
</head>
<body>
  <button onclick="clickCounter()">Haz clic</button>
  <p id="resultado"></p>

  <script>
    function clickCounter() {
      if (typeof(Storage) !== "undefined") {
        if (sessionStorage.clickcount) {
          sessionStorage.clickcount = Number(sessionStorage.clickcount) + 1;
        } else {
          sessionStorage.clickcount = 1;
        }
        document.getElementById("resultado").innerHTML =
          "Has hecho clic " + sessionStorage.clickcount +
          " veces en esta sesión.";
      } else {
        document.getElementById("resultado").innerHTML =
          "Tu navegador no soporta sessionStorage.";
      }
    }
  </script>
</body>
</html>
```

Explicación: `sessionStorage` se borra al cerrar la pestaña o el navegador, según el navegador. No se comparte entre pestañas. `localStorage` sí persiste.

Variantes: comparar ambos storages en una misma página; usar `sessionStorage.clear()`; guardar el contador en `localStorage` y la sesión en `sessionStorage`; probar en ventana privada.

---

8. UD3 — Ejercicios 3.13

8.1. Ejercicio 1: botón «volver» con `history.back()`

Enunciado. Crear un script que, mediante un botón, vaya a la pantalla anterior de navegación. Emplear `window.history.back()`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Volver</title>
</head>
<body>
  <button onclick="window.history.back()">Volver a la página anterior</button>
</body>
</html>
```

Explicación: `history.back()` equivale a `history.go(-1)`. Si no hay historial, el botón no hace nada. El PDF escribe `windows.history` (con s); el objeto correcto es `window.history`.

Variantes:

```
<button onclick="history.go(-2)">Volver dos páginas</button>
<button onclick="history.forward()">Avanzar</button>
<button onclick="history.go(0)">Recargar</button>
```

Otras variantes: comprobar `history.length` antes de volver; usar `window.location.href` para ir a una URL fija; confirmar antes de salir con `confirm()`.

8.2. Ejercicio 2: pop-up centrado con una página web

Enunciado. Crear un script que, al pulsar un enlace, abra un pop-up con una página web. Ayuda: `window.open("url", "nombre", "tamaño", "true/false")`. Ampliar: centrar el pop-up en la pantalla y abrir una nueva página web. Ayuda: `screen.width`, `screen.height`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Pop-up centrado</title>
</head>
<body>
  <a href="#" onclick="return abrirPopup()">Abrir pop-up centrado</a>

  <script>
    function abrirPopup() {
      let ancho = 700;
      let alto = 500;
      let izquierda = (screen.availWidth - ancho) / 2;
      let arriba = (screen.availHeight - alto) / 2;

      if (izquierda < 0) {
        izquierda = 0;
      }
      if (arriba < 0) {
        arriba = 0;
      }

      window.open(
        "https://www.example.com",
        "miPopup",
        "width=" + ancho + ",height=" + alto +
        ",left=" + izquierda + ",top=" + arriba +
        ",resizable=yes,scrollbars=yes"
      );

      return false;
    }
  </script>
</body>
</html>
```

Explicación: `window.open` debe ejecutarse dentro de un clic para que el navegador no lo bloquee. `left` y `top` se calculan con `screen.availWidth` y `screen.availHeight`, que descuentan las barras del sistema. `return false` evita que el enlace navegue a `#`. El PDF escribe `screen.heigh` (falta la t); la propiedad correcta es `screen.height`.

Variantes:

```
// Pop-up vacío y escribir contenido
let w = window.open("", "popup", "width=400,height=300");
w.document.write("<h1>Prueba</h1><p>Contenido generado con JavaScript.</p>");
w.document.close();

// Abrir una página local
window.open("noticia.html", "noticia", "width=600,height=400");

// Cerrar el pop-up desde la ventana principal
// w.close();
```

Otras variantes: abrir varias ventanas; comprobar si el navegador bloqueó el pop-up; centrar en el monitor actual; usar una página local en lugar de una URL externa.

8.3. Ejercicio 3: del 1 al 500, múltiplos de 3 y no primos

Enunciado. Recorrer un bucle del 1 al 500. Mostrar solo los múltiplos de 3 y el número de números que no son primos.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Múltiplos de 3 y no primos</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    function esPrimo(n) {
      if (n < 2) {
        return false;
      }
      for (let i = 2; i <= Math.sqrt(n); i++) {
        if (n % i === 0) {
          return false;
        }
      }
      return true;
    }

    let multiplosDe3 = [];
    let noPrimos = 0;

    for (let n = 1; n <= 500; n++) {
      if (n % 3 === 0) {
        multiplosDe3.push(n);
      }
      if (!esPrimo(n)) {
```

```
        noPrimos++;
      }
    }

    let lista = "";
    for (let i = 0; i < multiplosDe3.length; i++) {
      lista = lista + multiplosDe3[i];
      if (i < multiplosDe3.length - 1) {
        lista = lista + ", ";
      }
    }

    let html = "<p>Múltiplos de 3: " + lista + "</p>";
    html += "<p>Total de múltiplos de 3: " + multiplosDe3.length + "</p>";
    html += "<p>Números no primos entre 1 y 500: " + noPrimos + "</p>";
    document.getElementById("resultado").innerHTML = html;
  </script>
</body>
</html>
```

Explicación: Hay 166 múltiplos de 3 entre 1 y 500 y 405 números no primos (500 − 95 primos). El 1 no es primo. `Math.sqrt(n)` reduce las comprobaciones.

Variantes:

```
// Contar primos y no primos
let primos = 0;
for (let n = 1; n <= 500; n++) {
  if (esPrimo(n)) {
    primos++;
  }
}
console.log("Primos:", primos);
console.log("No primos:", 500 - primos);

// Mostrar también los no primos
let listaNoPrimos = [];
for (let n = 1; n <= 500; n++) {
  if (!esPrimo(n)) {
    listaNoPrimos.push(n);
  }
}
```

Otras variantes: múltiplos de 3 y 5; suma de múltiplos; criba de Eratóstenes; mostrar en tabla; usar `while`.

8.4. Ejercicio 4: menor y mayor de 10 números

Enunciado. Mostrar el número menor y mayor. Introducir 10 números enteros en un array y comprobar cada posición. Ampliación: introducirlos por teclado.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Menor y mayor</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    let numeros = [23, -5, 0, 18, 99, 3, 47, 8, -12, 56];

    let menor = numeros[0];
    let mayor = numeros[0];

    for (let i = 1; i < numeros.length; i++) {
      if (numeros[i] < menor) {
        menor = numeros[i];
      }
      if (numeros[i] > mayor) {
        mayor = numeros[i];
      }
    }

    document.getElementById("resultado").innerHTML =
      "Menor: " + menor + "<br>Mayor: " + mayor;
  </script>
</body>
</html>
```

Ampliación con `prompt`:

```
let numeros = [];
for (let i = 0; i < 10; i++) {
  let n;
  do {
    n = parseInt(prompt("Número " + (i + 1) + " de 10:"), 10);
  } while (isNaN(n));
  numeros.push(n);
}

let menor = numeros[0];
let mayor = numeros[0];
for (let i = 1; i < numeros.length; i++) {
  if (numeros[i] < menor) {
    menor = numeros[i];
  }
  if (numeros[i] > mayor) {
    mayor = numeros[i];
  }
}

document.getElementById("resultado").innerHTML =
  "Menor: " + menor + "<br>Mayor: " + mayor;
```

Explicación: Se inicializan `menor` y `mayor` con el primer elemento, no con 0, para que funcionen arrays con todos los números positivos o todos negativos. El mínimo y el máximo se calculan con comparaciones, sin funciones que no aparezcan en la teoría.

Variantes: mostrar la posición del menor y del mayor; calcular la media con un bucle; detectar duplicados con dos bucles; ordenar manualmente con el método de la burbuja; usar `while` en lugar de `for`.

8.5. Ejercicio 5: talla europea a Grande/Mediana/Pequeña

Enunciado. Identificar la talla de una prenda a partir de tallas europeas: `XXL`, `XL`, `L`, `M`, `S`, `XS`, `XXS`. La talla esperada es Grande, Mediana o Pequeña. Introducir por teclado y mostrar por pantalla.

Ayuda del PDF: Grande `{XXL, XL, L}`; Mediana `{S}`; Pequeña `{S, XS, XXS}`.

Aviso importante: la ayuda del PDF repite la `S` en Mediana y en Pequeña. Es una errata. Se proponen dos soluciones:

1. **Interpretación estándar:** Grande `{XXL, XL, L}`, Mediana `{M, S}`, Pequeña `{XS, XXS}`.
2. **Interpretación literal del PDF:** Grande `{XXL, XL, L}`, Mediana `{S}`, Pequeña `{S, XS, XXS}`; en este caso `S` se clasifica como Mediana porque se comprueba antes.

Solución estándar:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Tallas</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    let entrada = prompt("Introduce una talla (XXL, XL, L, M, S, XS, XXS):");

    if (entrada === null) {
      document.getElementById("resultado").innerHTML = "Operación cancelada.";
    } else {
      let talla = entrada.toUpperCase();
      let tipo;

      switch (talla) {
        case "XXL":
        case "XL":
        case "L":
          tipo = "Grande";
          break;
        case "M":
        case "S":
          tipo = "Mediana";
          break;
        case "XS":
        case "XXS":
          tipo = "Pequeña";
          break;
        default:
          tipo = "Talla no reconocida";
      }

      document.getElementById("resultado").innerHTML =
        "La talla " + talla + " es " + tipo + ".";
    }
  </script>
</body>
</html>
```

Solución literal con `switch`:

```
switch (talla) {
  case "XXL":
  case "XL":
  case "L":
    tipo = "Grande";
    break;
  case "S":
    tipo = "Mediana";
    break;
  case "XS":
  case "XXS":
    tipo = "Pequeña";
    break;
  default:
    tipo = "Talla no reconocida";
}
```

Explicación: Se normaliza a mayúsculas. El `switch` con casos agrupados es la solución más directa si se sigue el PDF literal. Si el usuario escribe espacios, se puede usar la función `quitarEspacios` de la sección 4.9 antes de `toUpperCase()`.

Variantes: incluir tallas numéricas (36, 38, 40...); usar `select`; validar antes de clasificar; mostrar una tabla de equivalencias; añadir tallas infantiles; usar `Map`.

8.6. Ejercicio 6: longitud de la circunferencia

Enunciado. Calcular la longitud \(L\) de la circunferencia dado su radio, usando objetos propios de JavaScript. Ayuda: \(L = 2 \pi r\).

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Longitud de la circunferencia</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    let radio = parseFloat(prompt("Introduce el radio:"));

    if (isNaN(radio) || radio < 0) {
      document.getElementById("resultado").innerHTML =
        "El radio debe ser un número no negativo.";
    } else {
      let longitud = 2 * Math.PI * radio;
      document.getElementById("resultado").innerHTML =
        "Longitud: " + (Math.round(longitud * 100) / 100) + " unidades.";
    }
  </script>
</body>
</html>
```

Explicación: `Math.PI` es la constante π. `Math.round(longitud * 100) / 100` redondea a dos decimales. Para el área se usa `Math.PI * Math.pow(radio, 2)`.

Variantes: calcular área y volumen de la esfera; pedir diámetro en lugar de radio; validar radio positivo; mostrar el resultado con 4 decimales; usar `Math.round`.

8.7. Ejercicio 7: Euromillones aleatorio

Enunciado. Generar aleatoriamente una apuesta de Euromillones: 5 números del 1 al 50 y 2 estrellas del 1 al 10. Ayuda: `Math.random()`, `Math.ceil()`, `Math.round()`, `array.push()`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Euromillones</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    function aleatorio(min, max) {
      return Math.floor(Math.random() * (max - min + 1)) + min;
    }
```

```
    function contiene(array, valor) {
      for (let i = 0; i < array.length; i++) {
        if (array[i] === valor) {
          return true;
        }
      }
      return false;
    }

    function generarCombinacion(cantidad, min, max) {
      let combinacion = [];
      while (combinacion.length < cantidad) {
        let numero = aleatorio(min, max);
        if (!contiene(combinacion, numero)) {
          combinacion.push(numero);
        }
      }
      return combinacion;
    }

    function textoArray(array) {
      let texto = "";
      for (let i = 0; i < array.length; i++) {
        texto = texto + array[i];
        if (i < array.length - 1) {
          texto = texto + " - ";
        }
      }
      return texto;
    }

    let numeros = generarCombinacion(5, 1, 50);
    let estrellas = generarCombinacion(2, 1, 10);

    document.getElementById("resultado").innerHTML =
      "Números: " + textoArray(numeros) +
      "<br>Estrellas: " + textoArray(estrellas);
  </script>
</body>
</html>
```

Explicación: `Math.random()` devuelve un número entre 0 (incluido) y 1 (excluido). La fórmula `Math.floor(Math.random() * (max - min + 1)) + min` genera un entero uniforme entre `min` y `max`. No se usa `Math.ceil(Math.random() * 50)` porque puede producir `0` cuando el aleatorio es exactamente 0. `contiene` recorre el array y evita repetidos sin usar `includes`, que no aparece en la teoría. No se ordena la combinación porque `sort` no aparece en la teoría.

Variantes:

```
// Con Math.ceil, controlando el 0
let n = Math.ceil(Math.random() * 50);
if (n === 0) {
  n = 1;
}

// Generar varias apuestas
let apuestas = [];
for (let i = 0; i < 5; i++) {
  apuestas.push(generarCombinacion(5, 1, 50));
}
```

Otras variantes: mostrar bolas con HTML y colores; simular sorteo con bombos; generar números sin repetir entre apuestas; comprobar que las estrellas no se repiten.

8.8. Ejercicio 8: matriz de filas × columnas

Enunciado. Introducir el tamaño de una matriz: número de filas `x` y número de columnas `y`. Crear los bucles necesarios para mostrar por pantalla las posiciones que contengan ambas coordenadas. Ejemplo: matriz 5 × 3; solución `(5,1) (5,2) ... (1,3) ... (5,3)`.

Solución:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Matriz de coordenadas</title>
</head>
<body>
  <div id="resultado"></div>

  <script>
    let filas = parseInt(prompt("Número de filas:"), 10);
    let columnas = parseInt(prompt("Número de columnas:"), 10);

    if (isNaN(filas) || isNaN(columnas) || filas <= 0 || columnas <= 0) {
      document.getElementById("resultado").innerHTML =
        "Las filas y las columnas deben ser enteros positivos.";
    } else {
      let html = "<table border='1' cellpadding='6'>";
      for (let i = 1; i <= filas; i++) {
        html = html + "<tr>";
        for (let j = 1; j <= columnas; j++) {
          html = html + "<td>(" + i + "," + j + ")</td>";
        }
        html = html + "</tr>";
      }
      html = html + "</table>";
      document.getElementById("resultado").innerHTML = html;
    }
  </script>
</body>
</html>
```

Explicación: El primer bucle recorre las filas y el segundo las columnas. Se generan todas las combinaciones, que es lo que significa «posiciones que contengan ambas coordenadas». El ejemplo del PDF es ambiguo en el orden; esta solución muestra `(1,1) (1,2) ... (filas,columnas)`. Si se quiere el orden del ejemplo, se invierte el recorrido de filas.

Variante solo texto:

```
let posiciones = "";
for (let i = 1; i <= filas; i++) {
  for (let j = 1; j <= columnas; j++) {
    posiciones = posiciones + "(" + i + "," + j + ") ";
  }
}
document.getElementById("resultado").innerHTML = posiciones;
```

Variantes: recorrer de `0` a `x-1`; mostrar solo la diagonal; mostrar solo los bordes; matriz con números aleatorios; suma de filas y columnas.

8.9. Ejercicio 9: calculadora completa

Enunciado. Completar el script de la calculadora. Implementar los botones de los números (0–9), las operaciones básicas (multiplicar, dividir, sumar, restar) y el botón de borrado. Añadir un cuadro secundario para visualizar el resultado anterior calculado, que se actualizará cuando se inicie una nueva operación. Ampliación: calcular la raíz cuadrada con `Math.sqrt(numero)`.

Solución completa:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Calculadora</title>
</head>
<body>
  <div id="anterior"></div>
  <input id="pantalla" value="0" readonly>

  <table border="1" cellpadding="8">
    <tr>
      <td><button onclick="limpiar()">C</button></td>
      <td><button onclick="raiz()">√</button></td>
      <td><button onclick="elegirOperador('/')">/</button></td>
      <td><button onclick="elegirOperador('*')">*</button></td>
    </tr>
    <tr>
      <td><button onclick="agregarDigito('7')">7</button></td>
      <td><button onclick="agregarDigito('8')">8</button></td>
      <td><button onclick="agregarDigito('9')">9</button></td>
      <td><button onclick="elegirOperador('-')">-</button></td>
    </tr>
    <tr>
      <td><button onclick="agregarDigito('4')">4</button></td>
      <td><button onclick="agregarDigito('5')">5</button></td>
      <td><button onclick="agregarDigito('6')">6</button></td>
      <td><button onclick="elegirOperador('+')">+</button></td>
    </tr>
    <tr>
      <td><button onclick="agregarDigito('1')">1</button></td>
      <td><button onclick="agregarDigito('2')">2</button></td>
      <td><button onclick="agregarDigito('3')">3</button></td>
      <td><button onclick="igual()">=</button></td>
    </tr>
    <tr>
      <td><button onclick="agregarDigito('0')">0</button></td>
      <td><button onclick="agregarPunto()">.</button></td>
      <td colspan="2"></td>
    </tr>
  </table>
```

```
<script>
  let actual = "0";
  let anterior = "";
  let operador = "";
  let reiniciar = false;

  function mostrar() {
    document.getElementById("pantalla").value = actual;
  }

  function agregarDigito(digito) {
    if (reiniciar) {
      actual = digito;
      reiniciar = false;
    } else {
      if (actual === "0") {
        actual = digito;
      } else {
        actual = actual + digito;
      }
    }
    mostrar();
  }

  function agregarPunto() {
    if (reiniciar) {
      actual = "0.";
      reiniciar = false;
    } else if (actual.indexOf(".") === -1) {
      actual = actual + ".";
    }
    mostrar();
  }

  function elegirOperador(op) {
    if (operador !== "" && !reiniciar) {
      calcular();
    }
    anterior = actual;
    operador = op;
    reiniciar = true;
  }

  function calcular() {
    if (operador === "") {
      return;
    }

    let a = parseFloat(anterior);
    let b = parseFloat(actual);
```

```
  let op = operador;
  let resultado;

  switch (op) {
    case "+":
      resultado = a + b;
      break;
    case "-":
      resultado = a - b;
      break;
    case "*":
      resultado = a * b;
      break;
    case "/":
      if (b === 0) {
        actual = "Error";
        document.getElementById("anterior").innerHTML =
          a + " / 0 = Error";
        operador = "";
        reiniciar = true;
        mostrar();
        return;
      }
      resultado = a / b;
      break;
  }

  actual = "" + Number(resultado.toPrecision(12));
  document.getElementById("anterior").innerHTML =
    a + " " + op + " " + b + " = " + actual;
  operador = "";
  reiniciar = true;
  mostrar();
}

function igual() {
  calcular();
}

function limpiar() {
  actual = "0";
  anterior = "";
  operador = "";
  reiniciar = false;
  document.getElementById("anterior").innerHTML = "";
  mostrar();
}

function raiz() {
  let numero = parseFloat(actual);
```

```
      if (numero < 0) {
        actual = "Error";
      } else {
        actual = "" + Math.sqrt(numero);
      }
      reiniciar = true;
      mostrar();
    }
  </script>
</body>
</html>
```

Explicación: La calculadora guarda el primer operando en `anterior`, el operador en `operador` y el número que se escribe en `actual`. `reiniciar` indica que el siguiente dígito debe empezar un número nuevo. El cuadro `#anterior` muestra la operación completa al calcular. La división entre cero muestra `Error`. La raíz cuadrada de un número negativo muestra `Error`. `toPrecision(12)` recorta los decimales que produce la representación interna de JavaScript; es un método del objeto `Number` que aparece en la teoría.

Variantes: añadir cambio de signo, porcentaje y memoria; permitir teclado con `onkeydown`; historial de operaciones; decimales con coma; modo científico con `Math.sin`, `Math.cos` y `Math.tan` si el profesor los admite; usar `BigInt` para enteros grandes.

8.10. Ejercicio 10: noticia con «Seguir leyendo»

Enunciado. Crear un script para insertar texto y poder seguir leyendo una noticia. El PDF proporciona un HTML con un párrafo truncado, un `p#demo1` y un botón que llama a `myFunction()`.

Solución fiel al PDF:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Noticias Páginas Web</title>
</head>
<body>
  <h2>Noticias Páginas Web</h2>

  <p>
    En el instituto, como ESCUELA PROMOTORA DE SALUD y dentro del proyecto de salud,
    llevamos varios años colaborando con Donantes de Sangre de Zaragoza. Esta
    colaboración consiste en facilitarles un espacio en nuestro...
  </p>

  <p id="demo1"></p>

  <button onclick="myFunction()">Seguir Leyendo</button>

  <script>
    function myFunction() {
      document.getElementById("demo1").innerHTML =
        "...centro, para que a cualquier persona de nuestra comunidad educativa " +
        "le resulte más accesible realizar su donación de sangre." +
        "<br>La siguiente donación será:" +
        "<br>Fecha: Martes 3 de octubre." +
        "<br>Lugar: Biblioteca del IES" +
        "<br>Horario: de 9:30 a 13:00 horas" +
        "<br>Recordad que TODOS EN CUALQUIER MOMENTO PODEMOS NECESITAR SANGRE";
    }
  </script>
</body>
</html>
```

Explicación: El párrafo truncado está en el HTML y el resto se inserta en `p#demo1` con `innerHTML` al pulsar el botón. Es la solución que pide el PDF: un botón, una función `myFunction()` y `document.getElementById("demo1").innerHTML`.

Variantes: guardar el texto completo en una variable y asignarlo; cargar la noticia desde un array de objetos; añadir varias noticias con un botón cada una; usar `innerHTML` también para el texto sin etiquetas.

---

9. UD3 — Ejercicios de ampliación 3.14

9.1. Ampliación 1: tabla de 8 × 8 con patrón

Enunciado. Crear una aplicación Web con JavaScript que cree una tabla de 8 filas por 8 celdas, rellenando cada una con la disposición de elementos de la imagen. La imagen muestra una tabla de 8 columnas con celdas alternas en gris y color crema, formando un damero, y sugiere métodos de creación de filas y celdas que no aparecen en la teoría.

Solución con `document.write`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Tabla 8 × 8</title>
</head>
<body>
  <script>
    document.write("<table border='1' cellspacing='0' cellpadding='0'>");

    for (let fila = 0; fila < 8; fila++) {
      document.write("<tr>");
      for (let columna = 0; columna < 8; columna++) {
        let color;
        if ((fila + columna) % 2 === 0) {
          color = "#fff8dc";
        } else {
          color = "#808080";
        }
        document.write(
          "<td style='width:45px;height:45px;background-color:" +
          color + "'></td>"
        );
      }
      document.write("</tr>");
    }

    document.write("</table>");
  </script>
</body>
</html>
```

Explicación: La tabla tiene 8 filas y 8 celdas. La expresión `(fila + columna) % 2` alterna dos colores y produce el damero. Los colores de la imagen son gris `#808080` y crema `#fff8dc`. El enunciado sugiere métodos de creación de filas y celdas que no aparecen en la teoría; `document.write` sí aparece y crea la misma tabla.

Variantes:

```
// Tabla con números del 1 al 64
document.write("<td style='width:45px;height:45px'>" +
               (fila * 8 + columna + 1) + "</td>");

// Tabla de multiplicar
document.write("<td>" + ((fila + 1) * (columna + 1)) + "</td>");

// Solo una diagonal
if (fila === columna) {
  color = "#808080";
} else {
  color = "#fff8dc";
}

// Tabla de tamaño variable
let tamano = parseInt(prompt("Tamaño de la tabla:"), 10);
```

Otras variantes: ajedrez con piezas; tabla con coordenadas; colores elegidos por el usuario; tabla creada con una cadena e `innerHTML`; tablero de ajedrez 8×8 con letras y números.

9.2. Ampliación 2: archivo `EjercicioAmpliacion2.7z`

Enunciado. Emplear el archivo `EjercicioAmpliacion2.7z`.

Estado: el archivo no está en el directorio, igual que `EjercicioAmpliacion1.7z`. No se puede resolver el enunciado concreto sin ese archivo. No se debe inventar la solución.

Método para resolverlo cuando se disponga del archivo:

1. Descomprimir con `7z x EjercicioAmpliacion2.7z` o con el gestor de archivos.
2. Abrir el HTML y localizar los `id`, clases, formularios y eventos.
3. Leer los comentarios `TODO` o las instrucciones del profesor.
4. Identificar qué objeto de JavaScript se pide: `Date`, `Math`, `String`, `Array`, `Object`, `JSON`, `Map`, `Set`, `Window`, `Document` o almacenamiento.
5. Implementar la lógica en un archivo `.js` separado si el enunciado lo pide.
6. Probar los casos límite: vacío, cero, negativo, texto no numérico, cancelar `prompt`.

Plantilla genérica:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>EjercicioAmpliacion2</title>
</head>
<body>
  <div id="resultado"></div>
  <script src="ejercicio2.js"></script>
</body>
</html>
```

```
// 1. Leer datos
// 2. Validar
// 3. Procesar con el objeto correspondiente
// 4. Mostrar el resultado
```

---

10. Catálogo de variaciones posibles

Esta sección sirve para cualquier cambio que haga el profesor sobre los enunciados. Todas las variaciones usan objetos y métodos de la teoría.

10.1. Cambios de entrada

Cambio	Cómo resolverlo
De `prompt` a `input`	Añadir `<input id="dato">` y leer `document.getElementById("dato").value`
De un valor a varios	Usar un `for`, `while` o un `array`
De números a texto	No usar `parseInt`; validar con `length`, `charAt()` y `charCodeAt()`
De positivo a cualquier entero	Usar `if (n < 0) { n = -n; }` o comparar con `0`
De 1 a 10 a otro rango	Cambiar los límites y validar con `>=` y `<=`
Cancelar `prompt`	Comprobar `if (valor === null)`
Entrada vacía	Comprobar `valor.length === 0`
Entrada no numérica	`isNaN(valor)` después de `parseInt` o `parseFloat`
Entero obligatorio	`isNaN(n) || Math.floor(n) !== n`

10.2. Cambios de salida

Cambio	Cómo resolverlo
`document.write` a un `div`	`document.getElementById("resultado").innerHTML = ...`
Texto a HTML	Usar `innerHTML` solo con contenido controlado
Consola a pantalla	Cambiar `console.log` por `innerHTML`
Una línea a varias	Unir con `" "`
Aviso a página	Cambiar `alert` por un `div` visible
Resultado anterior	Guardar en una variable y mostrarlo en otro elemento

10.3. Cambios de estructura

Cambio	Cómo resolverlo
`for` a `while`	Inicializar antes, condición y incremento dentro
`for` a `do...while`	El cuerpo se ejecuta al menos una vez
`for` a `for...of`	Para arrays y cadenas
`for` a `for...in`	Para recorrer índices o propiedades
`if` a `switch`	Útil para valores discretos; `switch (true)` para rangos

Cambio	Cómo resolverlo
`switch` a objeto	`let mapa = { clave: valor }; mapa[clave]`
Función suelta a función con parámetros	`function f(a, b) { return ...; }`
Función repetida a bucle	Mover la lógica dentro de un `for`

10.4. Cambios de DOM

Cambio	Cómo resolverlo
Crear tabla	`document.write()` con `<table>`, `<tr>` y `<td>`
Añadir fila	`document.write("<tr>...")` o concatenar en una cadena e `innerHTML`
Leer un campo	`document.getElementById("id").value`
Escuchar un clic	`onclick="funcion()"` en el HTML
Evitar recarga de enlace	`return false` al final de la función del `onclick`
Mostrar/ocultar	Guardar el texto en una variable y asignar `innerHTML` o `""`
Cambiar texto	`innerHTML`
Recorrer elementos	`getElementsByName()` o `getElementsByTagName()` con un `for`
Leer un formulario	`document.forms[0].elemento.value`

10.5. Cambios de almacenamiento

Cambio	Cómo resolverlo
Persistir al cerrar	`localStorage`
Borrar al cerrar sesión	`sessionStorage`
Guardar objeto	`JSON.stringify(objeto)` y `JSON.parse(texto)`
Borrar una clave	`removeItem("clave")`
Borrar todo	`clear()`
Contador inicial	`Number(localStorage.clave)	0`
Varias claves	Usar nombres distintos: `localStorage.contador1`, `localStorage.contador2`

10.6. Cambios de fechas

Cambio	Cómo resolverlo
Año actual	`new Date().getFullYear()`
Mes actual	`new Date().getMonth() + 1`
Día actual	`new Date().getDate()`
Día de la semana	`new Date().getDay()`
Fecha completa	Concatenar día, mes y año con `+`
Hora	`new Date().getHours()`, `getMinutes()`, `getSeconds()`

10.7. Cambios de aleatoriedad

Cambio	Cómo resolverlo
Entero entre `min` y `max`	`Math.floor(Math.random() * (max - min + 1)) + min`
Entero entre 1 y `max`	`Math.floor(Math.random() * max) + 1`

Cambio	Cómo resolverlo
Sin repetidos	Bucle `for` que compare con `===` antes de `push()`
Ordenado	Bucle manual de ordenación (método de la burbuja)
Varias apuestas	Repetir la función dentro de un `for`

10.8. Cambios de bits

Cambio	Cómo resolverlo
Dividir entre \(2^n\)	`x >> n` (entero con signo)
Multiplicar por \(2^n\)	`x << n`
Comprobar el resultado	Comparar con `Math.floor(x / Math.pow(2, n))`
Mostrar el binario	Bucle que divida entre 2 y guarde los restos

10.9. Cambios de validación

Caso	Código	
Entero	`isNaN(n)	Math.floor(n) !== n`
Número finito	`n !== Infinity && n !== -Infinity && !isNaN(n)`	
Rango	`n >= min && n <= max`	
Solo dígitos	Bucle con `charCodeAt(i)` entre `48` y `57`	
Letra	`charCodeAt(0)` entre `65` y `90`, o entre `97` y `122`	
DNI de 8 dígitos	`texto.length === 8` y `soloDigitos(texto)`	
Cadena no vacía	`texto.length !== 0`	
Cancelar prompt	`valor === null`	

---

11. Pruebas y errores típicos

11.1. Casos de prueba recomendados

Ejercicio	Casos
Par/impar	`0`, `2`, `-3`, `7`, texto, vacío
Múltiplo	`10` y `2`, `10` y `3`, divisor `0`, negativos
Factorial	`0`, `1`, `5`, `-1`, texto, número grande
Media	límites 4,5 / 5 / 7 / 9; notas fuera de 0–10
DNI	`12345678Z`, `87654321X`, `00000000T`, letra incorrecta, 9 dígitos, vacío
Tallas	`XXL`, `L`, `M`, `S`, `XS`, `xxs`, talla inventada
Euromillones	5 números únicos 1–50, 2 estrellas únicas 1–10
Matriz	1×1, 5×3, 0×3, texto
Calculadora	suma, resta, multiplicación, división, división por 0, raíz negativa, encadenar operaciones
Storage	recargar, cerrar pestaña, nueva pestaña, reiniciar

11.2. Errores típicos

Error	Causa	Solución
`NaN`	`parseInt` de texto no numérico	Validar con `isNaN`
`undefined`	Variable mal escrita o no inicializada	Revisar nombres; usar `let`
`null`	Usuario canceló `prompt`	Comprobar `=== null`
`document.write` borra la página	Se ejecuta después de cargar	Usar `innerHTML`
`Cannot read properties of null`	`getElementById` antes de que exista el elemento	Poner el script al final del `body`
Bucle infinito	No se incrementa la variable	Revisar `i++`
División por cero	No se valida	Mostrar error o comprobar `divisor !== 0`
`localStorage` no guarda	Navegador en modo restringido o clave mal escrita	Comprobar `typeof(Storage)`, usar la misma clave
Pop-up bloqueado	`window.open` fuera de un clic	Abrir dentro del evento del usuario
Acentos mal	Codificación	`<meta charset="UTF-8">`
Script no carga	Ruta incorrecta	Mismo directorio o ruta relativa correcta
Método no visto en teoría	Copiado de otro sitio	Sustituirlo por el equivalente de la sección 2.3

11.3. Cómo probar sin servidor

Abrir el HTML en el navegador con doble clic suele funcionar. Si el navegador bloquea `localStorage` en `file://`, usar un servidor local:

```
python3 -m http.server 8000
```

Y abrir `http://localhost:8000/archivo.html`.

Para ver errores: `F12` → Consola. Para probar funciones: pegar el código en la consola o usar `console.log`.

---

12. Checklist de entrega

- [ ] El archivo HTML tiene `<!DOCTYPE html>`, `lang="es"` y `meta charset="UTF-8"`.
- [ ] El archivo JS se llama exactamente como pide el PDF.
- [ ] No se han borrado líneas del código original cuando el enunciado lo prohíbe.
- [ ] No se han renombrado archivos ni variables pedidas.
- [ ] Solo se usan objetos y métodos de la teoría de UD2 y UD3.
- [ ] Los `prompt` cancelados no rompen la aplicación.
- [ ] Los valores no numéricos se validan con `isNaN`.
- [ ] Las divisiones por cero se controlan.
- [ ] Los resultados se muestran en el elemento correcto.
- [ ] Los bucles terminan.
- [ ] Los arrays se recorren con índices válidos.
- [ ] Se han probado los casos límite de la sección 11.
- [ ] La consola no muestra errores.
- [ ] El resultado coincide con el enunciado, no solo «funciona».

13. Mapa rápido de archivos

Archivo	Ejercicios
`op1.js`	2.12-1 a 2.12-7
`parimpar.html`	2.12-3
`factorial.html`	2.12-8
`Elección multiple-respuesta.html`	2.12-9
`op2.js`	2.13-1 a 2.13-5
`CalculoDNI.html` + `dni.js`	2.13-A2
`OperacionesBits.html` + `Operaciones.js`	2.13-A3
Página de almacenamiento	3.12-1 y 3.12-2
Páginas de 3.13	3.13-1 a 3.13-10
`EjercicioAmpliacion1`	3.14-1
`EjercicioAmpliacion2`	3.14-2 (archivo no disponible)

14. Resumen de resultados esperados

- Múltiplos de 3 entre 1 y 500: **166**.
- Números no primos entre 1 y 500: **405**.
- Primos entre 1 y 500: **95**.
- `40 >> 4` = **2**; `40 / 16` = **2,5**.
- `26 << 2` = **104**.
- `125 >> 3` = **15**; `25 >> 1` = **12**; `40 << 2` = **160**; `10 << 4` = **160**.
- `Number.MIN_VALUE` es el positivo más pequeño, no el más negativo.
- DNI válidos de ejemplo: `12345678Z`, `87654321X`, `00000000T`.
- Factorial: `0! = 1`, `5! = 120`.
- Euromillones: 5 números únicos del 1 al 50 y 2 estrellas únicas del 1 al 10.

Fin de la guía. Todas las soluciones usan exclusivamente los conceptos de las UD2 y UD3. Si el profesor pide un método que no aparece en la teoría, se sustituye por el equivalente de la sección 2.3.